Sitzung 3: Generierte Forschungssoftware

Dr. Julian Dehne

Gesellschaft für Informatik e.V. (GI) · deRSE
julian.dehne@gi.de · ORCID 0000-0001-9265-9619

RSE und generierter Code

Was heißt “generierte Forschungssoftware”?

- **Generierter Code:** Quellcode, den ein Sprachmodell ganz oder teilweise erzeugt hat, von der Zeilenvervollständigung bis zum agentisch erstellten Projekt.
- **Forschungssoftware:** Software, die im Forschungsprozess oder für einen Forschungszweck entstanden ist (Gruenpeter et al. 2021).
- **Generierte Forschungssoftware:** Beides zugleich. Der Code trägt ein Forschungsergebnis, und ein Teil davon stammt nicht von der Person, die das Ergebnis verantwortet.

Stand der Forschung

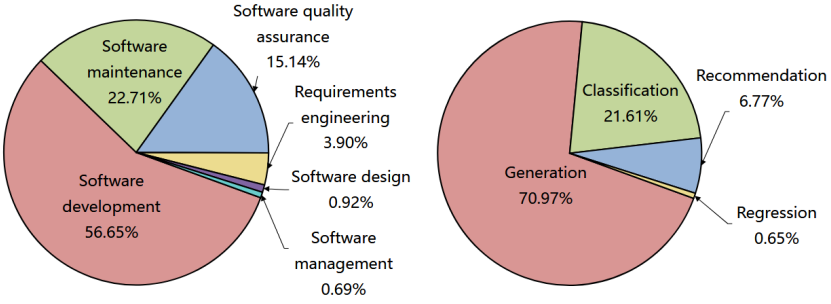
Sprachmodelle im Software Engineering

Systematische Literaturübersicht von Hou et al. (2024) über 395 Studien aus den Jahren 2017 bis Anfang 2024:

- Über die Hälfte der Studien betrifft die Softwareentwicklung (56,65 %), danach Wartung (22,71 %) und Qualitätssicherung (15,14 %).
- Anforderungen (3,90 %), Entwurf (0,92 %) und Management (0,69 %) kommen kaum vor.
- Rund 71 % der Aufgaben sind Generierungsaufgaben, allein 118 Studien zur Code-Generierung.

Die Forschung konzentriert sich auf das Erzeugen von Code. Die Phasen, in denen entschieden wird, *was* gebaut wird und *warum*, sind wenig untersucht.

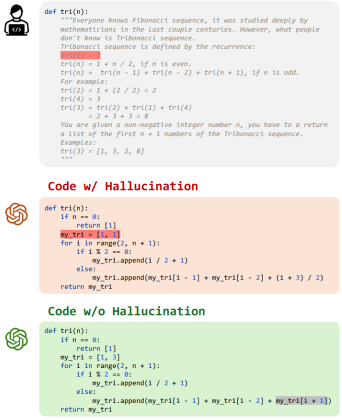
Wofür Sprachmodelle untersucht werden



(a) Distribution of LLM usages in SE activities. (b) Problem classification based on collected studies.

Figure 1: Verteilung der untersuchten Einsätze nach SE-Aktivität (a) und nach Problemtyp (b) (Hou et al. 2024).

Halluzinationen in generiertem Code



```
def tri(n):
    """Everyone knows Fibonacci sequence, it was studied deeply by
    mathematicians in the last couple centuries. However, what people
    don't know is Tribonacci sequence.
    Tribonacci sequence is defined by the recurrence:
    tri(1) = 3
    tri(n) = 1 + n / 2, if n is even.
    tri(n) =  tri(n - 1) + tri(n - 2) + tri(n + 1), if n is odd.
    For example:
    tri(2) = 1 + (2 / 2) = 2
    tri(4) = 3
    tri(3) = tri(2) + tri(1) + tri(4)
           = 2 + 3 + 3 = 8
    You are given a non-negative integer number n, you have to a return
    a list of the first n + 1 numbers of the Tribonacci sequence.
    Examples:
    tri(3) = [1, 3, 2, 8]
    """
```

Code w/ Hallucination

```
def tri(n):
    if n == 0:
        return [1]
    my_tri = [1, 3]
    for i in range(2, n + 1):
        if i % 2 == 0:
            my_tri.append(i / 2 + 1)
        else:
            my_tri.append(my_tri[i - 1] + my_tri[i - 2] + (i + 3) / 2)
    return my_tri
```

Code w/o Hallucination

```
def tri(n):
    if n == 0:
        return [1]
    my_tri = [1, 3]
    for i in range(2, n + 1):
        if i % 2 == 0:
            my_tri.append(i / 2 + 1)
        else:
            my_tri.append(my_tri[i - 1] + my_tri[i - 2] + my_tri[i + 1])
    return my_tri
```

Fig. 1. Illustration of code with or without hallucination.

Figure 2: Beispiel aus (Liu et al. 2026).

Liu et al. (2026): Was geht über funktionale Korrektheit hinaus schief?

- **Halluzination:** Die Ausgabe weicht von der Absicht ab, widerspricht sich selbst oder dem Weltwissen.
- Taxonomie aus generiertem Code: 3 Hauptkategorien, 12 Unterkategorien

Generierter Code und Open-Source-Lizenzen

Xu et al. (2025) (LiCoEval, ICSE 2025): Hält generierter Code die Lizenzen seiner Vorlagen ein?

- **Die Lizenz geht verloren:** Die Modelle geben Teile lizenzierten Open-Source-Codes fast wörtlich wieder, aber ohne die Angaben, an die die Wiederverwendung geknüpft ist.
- **Befund:** 14 Modelle, 4.187 Python-Funktionen. Bei den drei stärksten Modellen sind 0,88 bis 2,01 % der Ausgaben einer Vorlage „auffallend ähnlich“ (*striking similarity*).
- **Nachfrage hilft kaum:** Bei Copyleft-Code liegt die Trefferquote bei allen Modellen bis auf eines bei null.
- **Folge:** Das rechtliche Risiko liegt bei denen, die den Code verwenden.

Zwölf Tipps für KI-gestütztes Programmieren in der Wissenschaft

Bridgeford et al. (2026): *Twelve quick tips for AI-assisted coding in science*

1. Gather background knowledge before implementation
2. Define your problem and distinguish problem framing from coding
3. Choose appropriate AI interaction models
4. Think through and divide an implementation plan
5. Manage context strategically
6. Meticulously verify AI assistance for testing
7. Use tests to specify and constrain AI-generated code
8. Leverage AI to audit and expand test coverage
9. Monitor progress and know when to restart
10. Critically review generated code
11. Refine code incrementally with focused objectives
12. Document your code for reuse and publication

Weitere Befunde

Sicherheit, Produktivität, Reproduzierbarkeit

- **Unsicherer Code, sicheres Gefühl** (Perry et al. 2023): Teilnehmende mit KI-Assistent schrieben bei vier von fünf Aufgaben häufiger unsicheren Code und hielten ihn zugleich häufiger für sicher.
- **Erfundene Pakete** (Spracklen et al. 2025): 16 Modelle, 576.000 Codebeispiele. Mindestens 5,2 % (kommerzielle) bzw. 21,7 % (offene Modelle) der empfohlenen Pakete sind erfunden. Wer einen solchen Namen registriert, kann darüber Schadcode verteilen.
- **Gefühlt schneller, gemessen langsamer** (Becker et al. 2025): 16 erfahrene Entwickler:innen, 246 Aufgaben. Mit KI-Werkzeugen dauerten die Aufgaben 19 % länger; erwartet hatten sie 24 % Zeitersparnis.
- **Nicht ohne Nacharbeit lauffähig** (Vangala, Gehani, and Malik 2026): Von 300 durch Coding-Agenten erzeugten Projekten laufen nur 68,3 % ohne Nacharbeit.

Befunde aus der Wissenschaft: Code

- **Forschungscode ist schwerer** (Tian et al. 2024): SciCode, 80 Probleme aus 16 naturwissenschaftlichen Teilgebieten, von Forschenden erstellt. Das beste Modell (Claude 3.5 Sonnet) löste nur 4,6 % der Probleme.
- **Wie Forschende Sprachmodelle nutzen** (O'Brien 2025): oft, um sich in unbekannten Sprachen und Bibliotheken zurechtzufinden, also genau dort, wo sie die Ausgabe am wenigsten prüfen können.
- **Der "AI Scientist" im Test** (Beel, Kan, and Baumgart 2025): 42 % der Experimente scheiterten an Programmierfehlern.

Methodische Aspekte generierter Forschungssoftware

Vor der KI: Pfadgabelungen und viele Teams

- **Garten der Pfadgabelungen:** Ein Ergebnis hängt an einer Kette vertretbarer Entscheidungen (Ausschlüsse, Variablen, Modell, Berichterstattung). Jedes berichtete Modell steht für viele nicht gerechnete Alternativen (Bertran, Fogliato, and Wu 2026).
- **Multiplizität messen:** Many-Analysts-Studien halten Daten und Frage fest und lassen Teams variieren; Multiverse-Analysen rechnen die Varianten in einem Workflow durch. Offen bleibt, was als "vertretbar" gilt (Bertran, Fogliato, and Wu 2026).

Studie	Frage	Teams	Ergebnis
Silberzahn et al. (2018)	Zeigen Schiedsrichter Spielern mit dunkler Hautfarbe häufiger Rot?	29	Odds Ratios von 0,89 bis 2,93; 20 Teams fanden einen signifikanten Effekt, 9 nicht
Botvinik-Nezer et al. (2020)	Neun Hypothesen zu einem fMRT-Datensatz	70	Keine zwei Teams mit gleichem Workflow; deutlich verschiedene Ergebnisse
Breznau et al. (2022)	Senkt Zuwanderung die Unterstützung für Sozialpolitik?	73	Stark abweichende Ergebnisse; über 95 % der Varianz bleiben trotz Erfassung aller Entscheidungen unerklärt
Gould et al. (2025)	Zwei Fragen aus Ökologie und Evolutionsbiologie	174	Erhebliche Heterogenität der Effektstärken

Der agentische Garten der Pfadgabelungen

Miao, Pritchard, and Zou (2026): Reproduzieren KI-Agenten diese Streuung, wenn sie eine Vorannahme mitbekommen?

- **Aufbau:** Daten, Frage und Modell sind fest, nur die Persona mit ihrer Vorannahme ändert sich. Vier Fragen: Zuwanderung und Sozialstaat, Kaffee und Gesundheit, soziale Medien und psychische Gesundheit Jugendlicher, Darmflora und BMI.
- **Befund:** Agenten mit entgegengesetzten Vorannahmen ziehen aus denselben Daten entgegengesetzte Schlüsse. Sie erkunden andere Teile des Analyseraums und wählen am Ende bevorzugt die passende Analyse.
- **Wie bei Menschen:** In der Many-Analysts-Studie zur Zuwanderung mit 42 Teams reproduzieren die Agenten 72 % der ideologischen Lücke.
- **Nicht als fehlerhaft erkennbar:** 86 % der Berichte bestanden eine KI-Begutachtung, 78 % die Mehrheit menschlicher Gutachter:innen.

Agenten verstärken und verdecken die Auswahl

- **Altes Problem, neuer Maßstab:** Selektive Analysen werden mit Agenten viel leichter durchzuführen und viel schwerer zu entdecken (Miao, Pritchard, and Zou 2026).
- **Im Bericht unsichtbar:** Die Begutachtung sieht den gewählten Pfad, nicht die verworfenen.
- **Agentische Datenanalyse:** Agenten planen, führen Code aus, prüfen Zwischenergebnisse und überarbeiten. Wiederholter Zugriff auf dieselben Daten kann zu Überanpassung führen, auch wenn kein Einzelschritt fehlerhaft ist (Bertran, Fogliato, and Wu 2026).
- **Many Analysts mit Agenten:** Bertran, Fogliato, and Wu (2026) nutzen Agenten mit Personas als skalierbares Gegenstück zu verschiedenen Analyst:innen: rund 5.000 protokollierte Läufe (3 Datensätze, 4 Modelle, 5 Personas). Effektstärken und Schlüsse streuen mit Modell und Persona, auch bei methodisch gültigen Läufen.
- **Gegenvorschlag:** Protokollierte Agenten machen die Suche auch prüfbar. Der *m-value* gibt an, wie extrem ein Ergebnis unter vielen vertretbaren Analysen liegt; der *Agentic Bootstrap* schätzt ihn (Miao, Pritchard, and Zou 2026).

Literatur I

Becker, Joel, Nate Rush, Elizabeth Barnes, and David Rein. 2025. “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.” https://doi.org/10.48550/arXiv.2507.09089.

Beel, Joeran, Min-Yen Kan, and Moritz Baumgart. 2025. “Evaluating Sakana’s AI Scientist: Bold Claims, Mixed Results, and a Promising Future?” *ACM SIGIR Forum* 59 (1): 1–20. https://doi.org/10.1145/3769733.3769747.

Bertran, Martin, Riccardo Fogliato, and Zhiwei Steven Wu. 2026. “Many AI Analysts, One Dataset: Navigating the Agentic Data Science Multiverse.” *Proceedings of the National Academy of Sciences* 123 (29): e2606495123. https://doi.org/10.1073/pnas.2606495123.

Botvinik-Nezer, Rotem, Felix Holzmeister, Colin F. Camerer, Anna Dreber, Juergen Huber, Magnus Johannesson, Michael Kirchler, et al. 2020. “Variability in the Analysis of a Single Neuroimaging Dataset by Many Teams.” *Nature* 582 (7810): 84–88. https://doi.org/10.1038/s41586-020-2314-9.

Literatur II

Breznau, Nate, Eike Mark Rinke, Alexander Wuttke, Hung H. V. Nguyen, Muna Adem, Jule Adriaans, Amalia Alvarez-Benjumea, et al. 2022. “Observing Many Researchers Using the Same Data and Hypothesis Reveals a Hidden Universe of Uncertainty.” *Proceedings of the National Academy of Sciences* 119 (44): e2203150119. https://doi.org/10.1073/pnas.2203150119.

Bridgeford, Eric W., Iain Declan Campbell, Zijiao Chen, Zhicheng Lin, Harrison Ritz, Joachim Vandekerckhove, and Russell A. Poldrack. 2026. “Twelve Quick Tips for AI-Assisted Coding in Science.” *PLOS Computational Biology* 22 (7): e1014428. https://doi.org/10.1371/journal.pcbi.1014428.

Gould, Elliot, Hannah S. Fraser, Timothy H. Parker, Shinichi Nakagawa, Simon C. Griffith, Peter A. Vesk, Fiona Fidler, et al. 2025. “Same Data, Different Analysts: Variation in Effect Sizes Due to Analytical Decisions in Ecology and Evolutionary Biology.” *BMC Biology* 23 (1): 35. https://doi.org/10.1186/s12915-024-02101-x.

Gruenpeter, Morane, Daniel S. Katz, Anna-Lena Lamprecht, et al. 2021. “Defining Research Software: A Controversial Discussion.” Zenodo. https://doi.org/10.5281/zenodo.5504016.

Literatur III

Hou, Xinyi, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. “Large Language Models for Software Engineering: A Systematic Literature Review.” *ACM Transactions on Software Engineering and Methodology* 33 (8): 1–79. https://doi.org/10.1145/3695988.

Liu, Fang, Yang Liu, Lin Shi, Zhen Yang, Li Zhang, Xiaoli Lian, Zhongqi Li, and Yuchi Ma. 2026. “Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code.” *IEEE Transactions on Software Engineering* 52 (3): 1037–55. https://doi.org/10.1109/TSE.2026.3657432.

Miao, Jiacheng, Jonathan K. Pritchard, and James Zou. 2026. “The Agentic Garden of Forking Paths.” https://doi.org/10.48550/arXiv.2607.01507.

O’Brien, Gabrielle. 2025. “How Scientists Use Large Language Models to Program.” In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 1–16. https://doi.org/10.1145/3706598.3713668.

Perry, Neil, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. “Do Users Write More Insecure Code with AI Assistants?” In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2785–99. https://doi.org/10.1145/3576915.3623157.

Literatur IV

Silberzahn, Raphael, Eric L. Uhlmann, Daniel P. Martin, Pasquale Anselmi, Frederik Aust, Eli Awtrey, Štěpán Bahník, et al. 2018. “Many Analysts, One Data Set: Making Transparent How Variations in Analytic Choices Affect Results.” *Advances in Methods and Practices in Psychological Science* 1 (3): 337–56. https://doi.org/10.1177/2515245917747646.

Spracklen, Joseph, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. 2025. “We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs.” In *34th USENIX Security Symposium (USENIX Security 25)*, 3687–3706. https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen.

Tian, Minyang, Luyu Gao, Shizhuo Zhang, Xinan Chen, Cunwei Fan, Xuefei Guo, Roland Haas, et al. 2024. “SciCode: A Research Coding Benchmark Curated by Scientists.” In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 30624–50. https://doi.org/10.52202/079017-0963.

Vangala, Bhanu Prakash, Ashish Gehani, and Tanu Malik. 2026. “AI-Generated Code Is Not Reproducible (yet): An Empirical Study of Execution Reliability in LLM-Based Coding Agents.” In *Proceedings of the 4th ACM Conference on Reproducibility and Replicability (ACM REP ’26)*, 33–46. https://doi.org/10.1145/3820002.3828581.

Literatur V

Xu, Weiwei, Kai Gao, Hao He, and Minghui Zhou. 2025. “LiCoEval: Evaluating LLMs on License Compliance in Code Generation.” In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE 2025)*, 1665–77. https://doi.org/10.1109/ICSE55347.2025.00052.